# CSC 421 - Machine Learning 2

### Learning from Examples

## Instructor: Shengyao Lu

# Readings

CHAPTER 19 LEARNING FROM EXAMPLES

- Basic: Sections **19.1, 19.2, Summary**
- Expected: **19.3, 19.4, 19.6, 19.7.1, 19.7.5**
- Advanced: All the chapter including bibliographical and historical notes

Be sure you are familiar with the pseudocode. You may be tested on your understanding of it in the final.

# Last class

- **Supervised learning**: the agent observes input-output pairs $(X, Y)$ and learns a function that maps from input to output $f : X \to Y$.
  - **Classification**: the output is one of a finite set of values (such as sunny/cloudy/rainy or true/false).
  - **Regression**: the output is a number (such as tomorrow's temperature, measured either as an integer or a real number).
- **Unsupervised learning**: the agent learns patterns in the input without any explicit feedback.
  - clustering, self-supervised learning
- **Reinforcement learning**: the agent learns from a series of reinforcements: rewards and punishments.

| Terminology | Representation |
|---|---|
| Evidence | $D$ |
| Hypothesis $i$ | $h_i$ |
| Posterior | $P(h_i \mid D)$ |
| Likelihood | $P(D \mid h_i)$ |
| Prior | $P(h_i)$ |

1. $h_i \in \mathcal{H}$ refers to a hypothesis about the world, drawn from a hypothesis space $\mathcal{H}$.
2. In supervised learning the evidence $D = (X, Y)$; in unsupervised learning $D = X$.
3. In parameter learning the hypothesis $h_i = \theta_i$ refers to the model parameters; in structure learning $h_i = M_i$ refers to the network structure.

- **Bayesian learning**:

$$Posterior = \frac{likelihood \times prior}{Evidence}, \quad P(\theta_i \mid D) = \alpha P(D \mid \theta_i) P(\theta_i)$$

  - If $D = \{D_1, \ldots D_N\}$ is i.i.d., then

$$P(D \mid \theta_i) = \prod_j P(D_j \mid \theta_i)$$

  - Bayesian prediction for a new $D'$ (all hypotheses vote together)

$$P(D'|D) = \sum_{\theta_i} P(D'|\theta_i) P(\theta_i|D)$$

- **MAP approximation**: vote with the best hyposthsis

$$\text{with } \log,\ \theta_{\text{MAP}} = \arg\max_{\theta_i}\ \log P(D \mid \theta_i) + \log P(\theta_i)$$

- **Maximum likelihood Estimation (MLE)**: vote with the best hyposthsis, and without prior

$$\text{with } \log,\ \theta_{\text{MLE}} = \arg\max_{\theta_i}\ \log P(D \mid \theta_i)$$

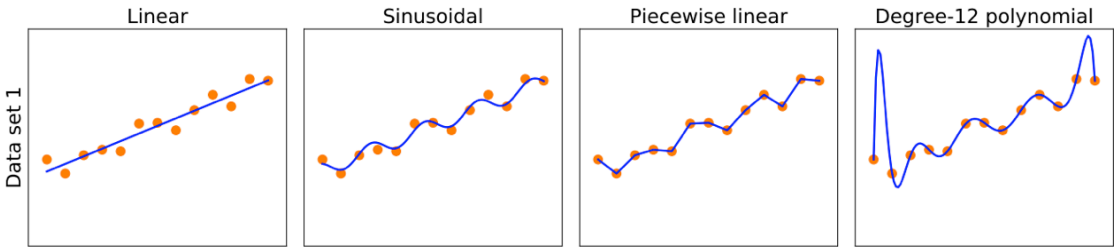
- Benefits of using $\log$: turns products into sums, easier to differentiate, easier to optimize, numerical stability (avoids underflow from multiplying many small probabilities issues.)
- **EM algorithm**: With hidden variables $Z$

$$\theta^{(t+1)} = \arg\max_{\theta}\ \mathbb{E}_{Z|X,\theta^{(t)}}\left[\log\ P(X, Z \mid \theta)\right]$$

# 3. Supervised learning

- training set: $(x_1, y_1), \ldots, (x_N, y_N)$, where $y_i$ is ground truth.
- What we want is a consistent hypothesis $h$ such that each $x_i$ in the training

set has $h(x_i) = y_i$.
  - or a best fit such that $h(x_i)$ is close to $y_i$.
- test set: $(x_1, y_1), \ldots, (x_M, y_M)$.
  - We say $h$ generalizes well if it accurately predits the outputs of the test set.




- Learning Tasks: classification, regression
- Learning models:
  - Linear Models: Linear Regression, Logistic Regression, Perceptron Learning
  - Distance-Based Models: k-Nearest Neighbour
  - Margin-Based Models: Support Vector Machine (SVM)
  - Tree-Based Models: Decision Tree, Random Forest
- Learning Principles:
  - Optimization: Loss Functions, Gradient Descent
  - Generalization: bias, overfitting, regularization

## 3.1 Learning Principles

Supervised learning is usually formulated as **optimization** problems, but grounded in probability and statistics, where you initialize $\theta$ to some values, then keep updating them.

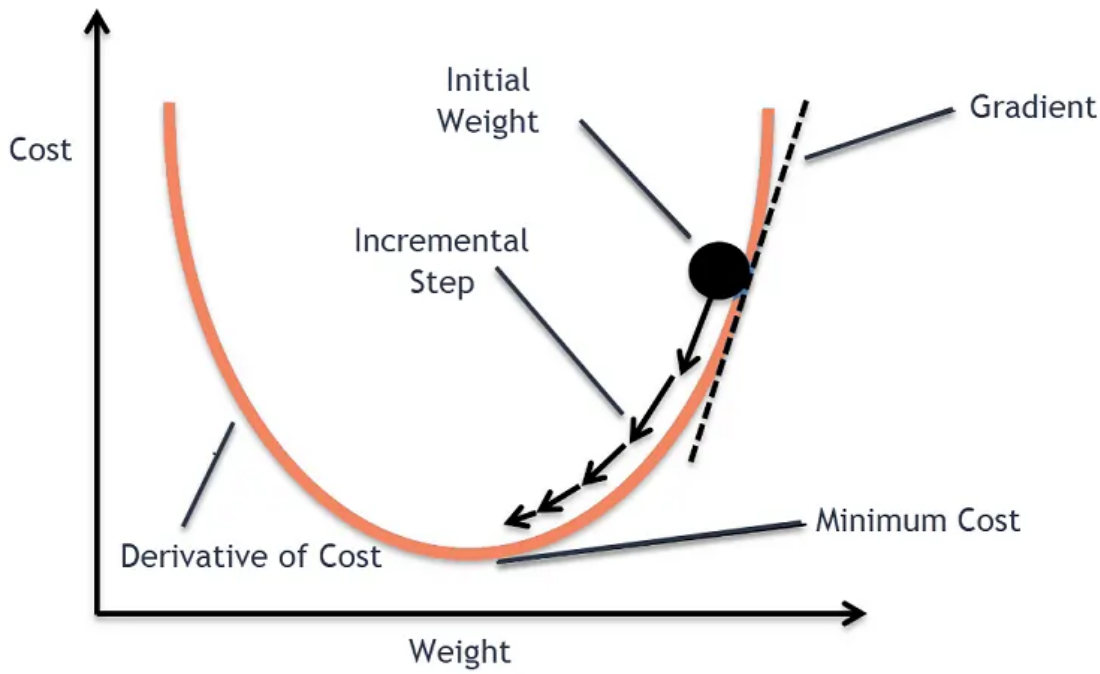
- **Loss function** $\mathcal{L}$: A function that measures how different a prediction is from the true value. The larger the error, the larger the loss.
- **Bias**: Error caused by overly strong assumptions in the model (model is too naive).
  - High Bias = Underfitting
- **Overfitting**: A model fits the training data so well that it fails to generalize to test data sets (model is too complex).
  - Overfitting = Poor generalization
- **Regularization**: A term added to the loss function to penalize overfitting.
  - regularized loss = data loss + strength · regularization

> **Recall:**
>
> - In statistical learning, we use MLE and MAP to formulate probabilistic learning as optimization problems.
>
>  > - MLE: maximize log-likelihood
>  > - MAP: maximize (log-likelihood + log-prior)
>  > - Here, negative log-likelihood can be viewed as a loss function, and negative log-prior can be viewed as a regularization term.

- **Gradient descent**: keep updating parameters $\theta$ to reduce loss on training data.

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$$

  - Take derivative of the loss function w.r.t. $\theta$, update $\theta$ by learning rate $\eta \cdot$ gradient of $\mathcal{L}(\theta)$.




## Typical Loss functions in supervised learning

- Regression task (for N data samples in $(X, Y)$).
  1. Mean Squared Error (MSE, aka squared $L_2$ loss): smooth and easy to optimize

$$\mathcal{L}(Y, \hat{Y}) = \frac{1}{N} \sum_{i}^{N} (y_i - \hat{y}_i)^2$$

2. Mean Absolute Error (MAE, aka $L_1$ loss): robust to outliers

$$\mathcal{L}(Y, \hat{Y}) = \frac{1}{N} \sum_{i}^{N} |y_i - \hat{y}_i|$$

- Classification task
  1. 0-1 Loss Function: straightforward, but not differntiable

$$\mathcal{L}(y, \hat{y}) = \begin{cases} 0 & y = \hat{y} \\ 1 & y \neq \hat{y} \end{cases}$$

  2. Binary Cross-Entropy Loss (BCE)

$$\mathcal{L}(y, \hat{p}) = -[y \log \hat{p} + (1 - y) \log(1 - \hat{p})]$$

  3. Softmax Loss: $p_y$ is the predicted probability of the true class.

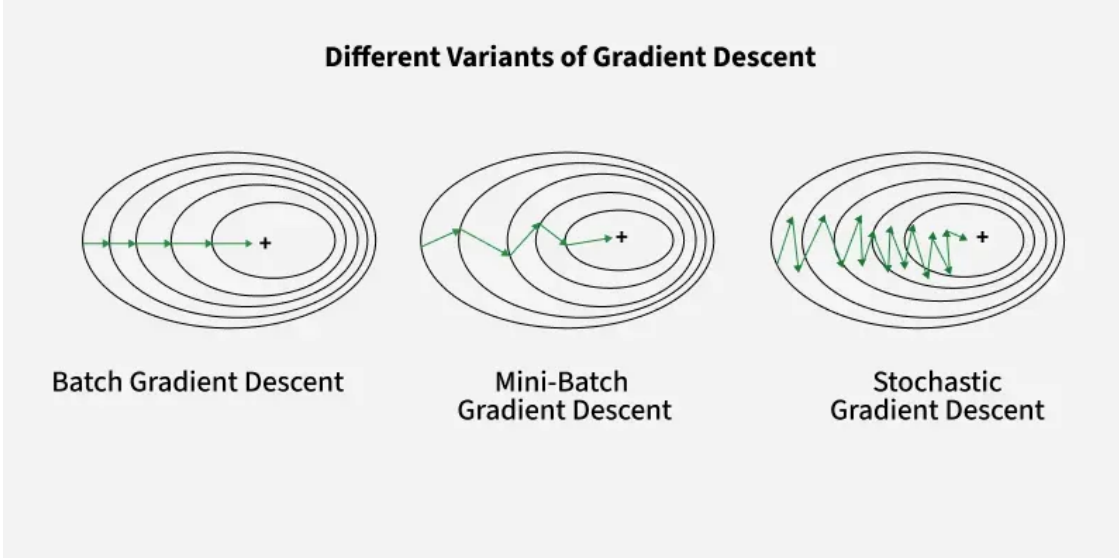
$$\mathcal{L} = -\log p_y$$

## Typical regularizations

- Parameter Norm Penalties
  1. L2 Regularization（Ridge, squared L2-norm） $\|\theta\|_2^2$
  2. L1 Regularization (Lasso, L1-norm) $\|\theta\|_1$

## Typical Gradient Descent methods

| Method | Data Used per Update | Update Stability | Computational Efficiency |
|---|---|---|---|
| Batch GD | Entire dataset | High | Slow |
| SGD | One sample | Low (noisy) | Fast |
| Mini-batch GD | Small batch | Medium | Most efficient |




## 3.2 Linear models

- Linear regression: use a line or Hyperplane to describe the relationship between the inputs and outputs.
  - good for regression task.
  - $\theta = \begin{bmatrix} b \\ \mathbf{w} \end{bmatrix}$.
  - Typically use MSE as the loss.

$$\hat{y} = \mathbf{w}^\top X + b$$

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}$$

- Logistic Regression:
  - good for binary classification
  - usually use BCE as the loss function
  - may overfit with high-dimensional features, so L2 regularization is often used

$$\hat{p} = \sigma(z), \quad z = \mathbf{w}^\top X + b$$

$$\sigma = \frac{1}{1 + e^{-z}}$$

$$\hat{y} = \begin{cases} 0 & \hat{p} < 0.5 \\ 1 & \hat{p} \geq 0.5 \end{cases}$$

- Perceptron learning
  - good for binary classification
  - update $\theta$ when wrongly classified a data sample

$$\hat{y} = \text{sign}(\mathbf{w}^\top X + b)$$

## 3.3 Decision Tree

```
if age < 30:
    if income > 50k:
        class = buy
    else:
        class = not buy
else:
    class = buy
```

- A decision tree **learns a set of if–then rules** to classify data.
  - see textbook for the pseudocode.
  - Terminology:
    - Root Node: Starting point representing the whole dataset.
    - Branches: Lines connecting nodes showing the flow from one decision to another.
    - Internal Nodes: Points where decisions are made based on data features.
    - Leaf Nodes: End points of the tree where the final decision or prediction is made.
  - Choose the feature with the most information gain (based on entropy).
    - $p_i$ = probability of class i
    - $k$ = number of classes
    - $S$ = current dataset
    - $A$ = feature used for splitting
    - $S_v$ = subset of S where feature A takes value v
      $|S|$ = number of samples in S
    - $$H(S) = -\sum_{i=1}^{k} p_i \log_2 p_i$$

$$IG(S, A) = H(S) - \sum_{v} \frac{|S_v|}{|S|} H(S_v)$$

  - pros:
    - represent non-linear relationships, e.g., if $x_1 > 5$ and $x_2 < 3$.
    - not sensitive to feature scale, e.g., age = 30, income = 100000, in

DT, we only have comparison like income > 5000
    - naturally handles discrete R.V., e.g., color={red, blue, green}
    - strong explainability
  - cons:
    - tree can be deep, and hence easy to overfit
    - decision boundary is axis-aligned and piecewise constant
    - training is greedy and may not find the globally optimal tree

## 3.4 k-Nearest Neighbour (kNN)

1. Parametric model: learns a set of fixed size parameters $\theta$, no matter how many training examples are there.
2. Nonparametric model: retains data points as part of the model, kNN is one of such models.

- **kNN**: The prediction for a query point $x_q$ is determined by its k nearest neighbors. $NN(k, x)$ finds the set of $k$ neighbours.
  - instance-based learning
  - no explicit model training
  - During "training phase", it simply store the training data points $D = \{(x_i, y_i)\}_{i=1}^n$.
  - Prediciton complexity: scan all the training data.
- **kNN** prediction formula
  - classification: majority vote

$$\hat{y} = \arg\max_c \sum_{i \in NN(k, x_q)} \mathbf{1}(y_i = c)$$

  - regression

$$\hat{y} = \frac{1}{k} \sum_{i \in NN(k, x_q)} y_i$$

- **Distance metric**
  - Euclidean distance

$$d(x_i, x_j) = \sqrt{\sum_{m=1}^{d} (x_{im} - x_{jm})^2}$$

  - Manhattan distance

$$d(x_i, x_j) = \sum_{m=1}^{d} |x_{im} - x_{jm}|$$

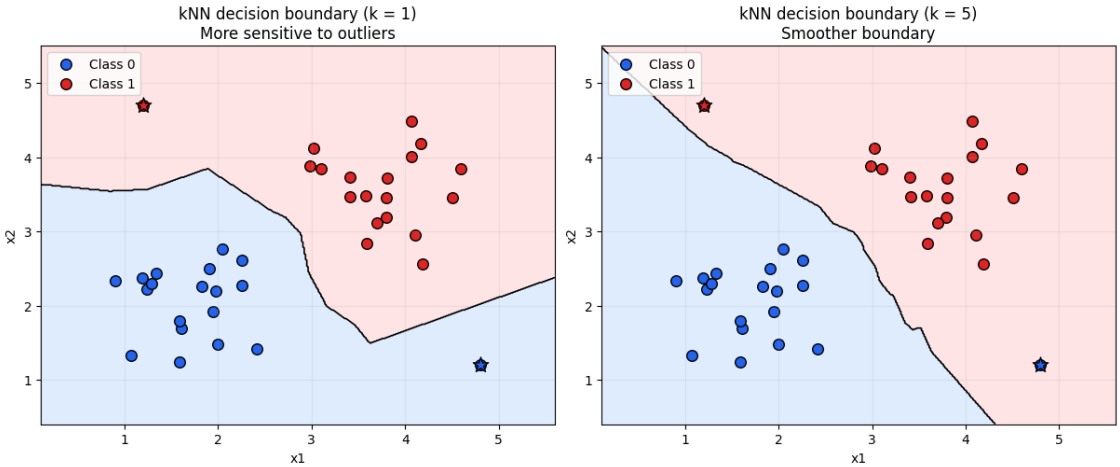
- Minkowski distance

$$d(x_i, x_j) = \left( \sum_{m=1}^{d} |x_{im} - x_{jm}|^p \right)^{1/p}$$

- sensitive to feature scale, therefore normalization is used:

$$x' = \frac{x - \mu}{\sigma}$$

- selection of $k$: cross-validation
  - low k: low bias, high variance, can be overfit
  - high k: high bias, low variance, can be underfit
- pros: naturally nonlinear, can fit complex decision boundary
- cons: memory expensive, prediction slow, curse of dimensionality, sensitive to noise




## 3.5 Support Vector Machine (SVM)

- **SVM:** find the hyperplane that maximizes the margin. use kernel trick to realize nonlinear classification.
  - SVMs construct a **maximum margin separator**—a decision boundary with the largest possible distance to example points.
  - SVMs create a **linear separating hyperplane**, and **embed the data into a higher-dimensional space**, and use the kernel trick to avoid explicitly computing the mapping.

        - Often, data that are not linearly separable in the original input space.
        - However they are often easily separable in the higher-dimensional space.
    - SVMs are nonparametric—the separating hyperplane is defined by a set of example points, not by a collection of parameter values.
        - Unlike kNNs that need to retain all the examples, an SVM model keeps only the examples that are closest to the separating plane—usually only a small constant times the number of dimensions.
        - **"Some data points are more important than others."**
- Pros: generalize well, flexibility to represent complex functions, resistant to overfitting.
- Cons: training expensive $O(n^2)$ to $O(n^3)$, kernel selection difficult
- Linear separator: hyperplane $w^\top x + b = 0$
- Prediction: $\hat{y} = \text{sign}(w^\top x_q + b)$
- Margin: two times the distance from the separator to the nearest point
    - distance from data sample to hyperplane

$$\frac{|w^\top x_i + b|}{\|w\|}$$

- Support vector: data points closest to the separator

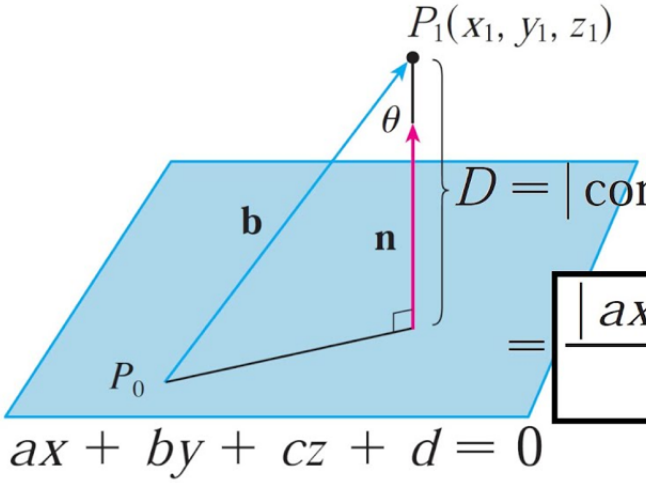
When data are linearly classifiable, and we assume support vectors satisfy $y_i(w^\top x_i + b) = 1$, it becomes an optimization problem

$$\min_{w,b} \frac{1}{2}\|w\|^2, s.t., y_i(w^\top x_i + b) \geq 1$$

When data are not linearly classifiable, we use a feature mapping $\phi(x)$, and the hyperplane becomes $w^\top \phi(x) + b = 0$. Or, we rewrite this optimization to Lagrangian, and derive the dual form, we can use kernel (inner product) trick, e.g., RBF kernel, to map features to a high dim space, and use $K(d_1, d_2) = \phi(d_1)^\top \phi(d_2)$ to compute inner products in the feature space, without explicitly calculating $\phi(d)$.

Aside:

# Distance from a Point to a Plane




$$D = |\text{comp}_{\mathbf{n}}\,\mathbf{b}| = \frac{|\mathbf{n}\cdot\mathbf{b}|}{|\mathbf{n}|} = \boxed{\frac{|ax_1 + by_1 + cz_1 + d|}{\sqrt{a^2 + b^2 + c^2}}}$$

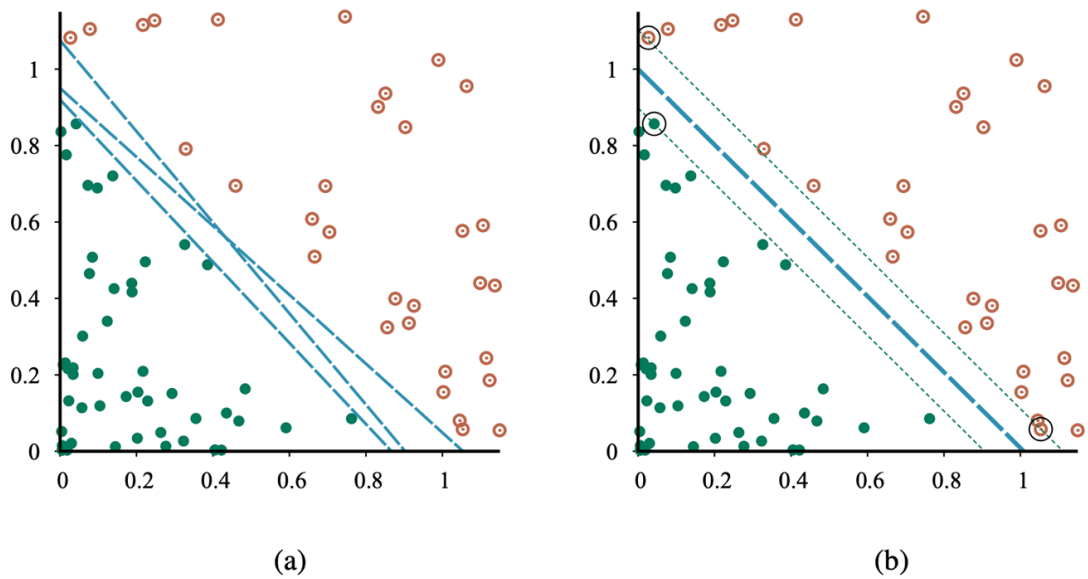



**Figure 19.21** Support vector machine classification: (a) Two classes of points (orange open and green filled circles) and three candidate linear separators. (b) The maximum margin separator (heavy line), is at the midpoint of the **margin** (area between dashed lines). The **support vectors** (points with large black circles) are the examples closest to the separator; here there are three.